

```
#include <stdio.h>
#include <sys/sdt.h> /* Include this header file */

int main(void)
{
    printf("In main\n");

    STAP_PROBE(test, in_main);

    return 0;
}
```

Using them is fairly simple. Also, compilation doesn't involve any tricks. Just the same old:

```
$ gcc test.c -o test
```

Let's probe and trace the markers.

Probing and tracing markers with Systemtap

Using SDT markers won't give us anything unless we probe and trace them. For this, we need a tool called Systemtap.

Systemtap, developed by RedHat, is a very powerful debugging and performance analysis tool that's widely used these days. You need to install the packages *systemtap* and *systemtap-runtime*. In Fedora and RedHat-based systems, run the following command:

```
yum install systemtap systemtap-runtime
```

If you face any problems with the installation and usage, go to the site: <https://sourceware.org/systemtap/> and access the beginners' guide, which shouldn't be very hard to follow. User space probing functionality takes help from the *uprobes* infrastructure which has been present in the kernel since kernel version 3.5. To check your kernel version, use the following command:

```
$ uname -r
```

And to check if the *uprobes* functionality is available, use:

```
$ grep CONFIG_UPROBES /boot/config-$(uname -r)
```

...which should show you the following:

```
CONFIG_UPROBES=y
```

After you make sure that Systemtap is installed, open another terminal to run *stap*.

In one terminal, issue the following code:

```
# stap -v -e 'probe process("/home/user/test").mark("in_
```

Figure 1: *stap* waiting for the marker to be hit

Figure 2: *test* runs three times and marker was hit three times

```
main") { printf("Marker in_main Hit\n"); }
```

What we have done here is run a Systemtap script from the command line mentioning that we need to probe executable with path as */home/user/test*, and the marker we need to probe into is *in_main*. And if control hits the marker, we must display *Marker in_main Hit*. The option *-e* is to instruct *stap* that we are providing the script on the command line itself and *-v* is to display a verbose output.

Run this command and you will find that after five passes (refer to Figure 1), it just waits—for the program to be run and the marker to be hit. So, in another terminal, run the program with the marker and see the result in the terminal running Systemtap (refer to Figure 2).

How does this help? You can place these markers (any number of them) in programs to determine the code flow. One can argue that it would be much simpler to use *printf()* instead of these markers to determine the code flow. But every time you run the program, a lot of unnecessary data is displayed. We don't want that, nor do we want to interfere with the program's output. We want the data to be displayed only when we need it. SDT markers are dormant. They can never do anything on their own. Only when somebody needs more information, the markers can be probed with tools like Systemtap to find out the details.

SDT markers with arguments

Only finding out the hits on SDT markers isn't enough.